

Supplementary Information (v3): Forecast skill of structural models for Northern cod — a scored ladder

Amin Abaee

Independent Researcher

ORCID: [0000-0002-0019-1842](https://orcid.org/0000-0002-0019-1842) Email: amin_abaee@ut.ac.ir

September 18, 2026

typesetting build: supp3-c1375ed

Forecast skill of structural models for Northern cod: a scored ladder

This supplement supports the main article (carrier of record: paperE1_cod_forecast_ladder_v56_humanized, stamp hu-c1375ed; prior carriers v49–v51 unchanged by this supplement). Section numbers are cited in the main text as SI-1, SI-2, and so on. All content here is derived from the analysis code and result files archived with this study; no result appears in this supplement that is not reproducible from that code.

SI-1 Order in which the passes were run

The main text (Section 4) states that the study is a fixed computational protocol rather than a prospective registration, that no dated pre-scoring protocol file exists, and that the additional passes were declared in the text as they were added. This section records the order of execution, documenting which choices were fixed before scoring and which were not.

The order below is established from the analysis code and its recorded metadata, not from recollection. Passes 1 and 2 are labelled as such in the driver script `run_ladder.py` and are distinguished in the archived `results/meta.json` by the keys `catch_pass1` and `catch_pass2`. Passes 3 and 4 are separate scripts that import from the pass-1/2 driver `run_ladder.py`, which fixes their order after it: `run_xte.py` imports the model definitions (SPECS) themselves, while the capelin scripts import the shared machinery (DATA, OUT, EPS, step, naive_baselines) and add one further model of their own, `M_cap_index`.

#	Pass	Catch / data treatment	Produced by	Archived output
1	Regime catch	Regime constants 240/120/5 kt, read from the SAR prose	run_ladder.py (# Pass 1 (regime C))	fixed_window_scores.csv, rolling_summary.csv (rows catch = regime)
2	Annual landings	Annual series, Schijns et al. (2021) Table 1, tonnes/1000	run_ladder.py (# Pass 2: annual catch)	same files, rows catch = annual
2b	Survey start	Forecast initiated at the survey state, annual catch	run_survey_start.py called within the pass-2 block of run_ladder.py	survey_start_forecasts.csv
3	Capelin	Continuous capelin-W ablation, observed acoustic years only	run_capelin_index.py, run_capelin_regime.py	capelin_index_summary.csv, capelin_regime_summary.csv
4	Specification B	The ladder on the xteNCAM domain alone, not pooled with NCAM 2016	run_xte.py	xte_rolling_summary.csv, xte_fixed_window_scores.csv

Three points follow from this order and bear on how the scores should be read.

The five ladder models were fixed before any pass was scored. The SPECS list in `run_ladder.py` defines M1, M1b, M2, M3 and M4 once, and Specification B (pass 4) imports that same list rather than redefining it. None of M1–M4 was altered after a score was seen. The capelin pass is the one place where a model was added after scoring began: it introduces `M_cap_index`, a prey-informed variant, which is reported in the main text as its own ablation and is not part of the five-rung ladder.

Two elements of Definition 2.4 were formalised after the Section 3 scores had been computed: the 5% tie band, and the comparator declarations for the non-nested rungs. This is the substantive qualification on the protocol, stated in the main text. Neither element changes any retention outcome: the tie band never decides a retention result, the smallest deficit being 17%.

Specification B was run last and was never pooled with the NCAM 2016 domain. The constraint is recorded in the code itself, whose module docstring reads “Wave E ladder on Ω_{xte} alone. Do not pool with NCAM 2016.”, and in `meta.json`, which records that the full `xteNCAM` SSB table was not extracted and was not pooled. The two specifications are therefore scored separately throughout, and no claim in the main text transfers a result from one to the other.

SI-2 Schaefer is not the (s) limit of the depensation family

Statement. In the family of Definition 2.1, the Schaefer law corresponds to the choice (a). The limit (s) in the depensation branch yields $(a(S_t) = S_t/K)$, hence a cubic modification of the logistic surplus, and not the Schaefer law.

Proof. Substituting $(s = 0)$ in $(a(S_t) = (S_t - s)/(K - s))$ gives $(a(S_t) = S_t/K)$, so $(g(S_t) = r S_t (1 - S_t/K)(S_t/K))$, a cubic surplus term. The Schaefer law requires (a), which is the separate branch in which the depensation factor is replaced, not obtained by a limiting value of (s). ■

The distinction matters because it fixes what M1b adds to M1: a separate branch of the family, not a limiting case of it. Abaee (2026b) makes the same distinction.

SI-3 Uncertainty layer: bandwidth grids, disagreement rows, and provenance

Bandwidth and block-length sensitivity

The M1-against-persistence margin was recomputed over a grid of HAC truncation lags and moving-block bootstrap block lengths, because the automatic $(h-1)$ truncation and the $((h,3))$ block length are not neutral choices when origins overlap at $(h=5)$, the estimation window expands, the predictand is a smoothed reconstruction, and the sample straddles a structural break.

Quantity	Specification A	Specification B
DM (z), $(h=1)$, lag 0 $\rightarrow$ lag 3	1.14 $\rightarrow$ 2.30	2.81 $\rightarrow$ 1.70
Bootstrap (p), $(h=1)$, block 3–9	0.17–0.19	0.036–0.049
Bootstrap (p), $(h=5)$, block 3–9	0.26–0.28	0.008–0.026

The DM statistic crosses the conventional 1.96 threshold within the range of defensible bandwidths on both specifications — upward on A, downward on B — so no verdict rests on it. The bootstrap is markedly more stable. The qualitative reading in the main text (Specification A within noise, Specification B separating) is therefore a bootstrap result rather than a DM result. The grid was run for the M1-against-persistence margin; robustness across block lengths is established for that comparison and is not asserted for every module.

The four disagreement rows

Four of the twenty-eight rows of Table 9 have a bootstrap interval excluding zero while $(|z| < 1.96)$:

Spec	(h)	Module	Comparator	DM (z)	95% CI (kt)
A	1	M4	M3	0.99	[+4.7, +144.7]
B	1	M3	persist	1.85	[+1.0, +92.5]
B	5	M1b	persist	1.80	[+18.2, +250.9]
B	5	M4	M3	1.88	[+20.2, +177.4]

Both summaries are reported side by side and no verdict rests on either where they disagree.

Provenance

The layer is produced by `campaign_e1_dm_uncertainty.py`, deterministic under seed 0, with output archived at `results/e1_dm_uncertainty.csv`.

SI-4 Log-floor binding counts

The log-RMSE scores of Table 4 use $((S, \{f\}))$ with $(\{f\} = 10^{\{-3\}})$ kt, the trajectory code clipping the state to $([_ \{f\}, 10^{\{6\}}])$ kt. The floor binds on the following counts of origins, taken from the archived per-origin records (Specification A: the annual-landings rolling pass of Section 3.2).

Spec	(h)	origins	M1	M1b	M2, M3, M4
A	1	25	15	17	M3 at 3; M2 and M4 at most once per horizon
A	5	21	19	19	M3 at 3; M2 and M4 at most once per horizon
B	1	59	22	24	between 0 and 11
B	5	55	36	46	between 0 and 11

The raw-RMSE column, not the log column, is the retention score, so these counts qualify a reported diagnostic rather than the verdict.

SI-5 Retention-rule operating characteristics: full decomposition

Design and results are archived in full as E1_SIMULATION_RESULTS.md, with the pre-registration at wave_e_cod/SPECIFICATION_v4.md (locked before execution) and the per-replicate output at wave_e_cod/results/sim_retention_power.csv (10,000 rows). This section records the detail condensed out of Section 3.7.

SI-5.1 Where the power is lost

H2 alone is “beats persistence by more than the 5% band at both horizons”, ignoring the comparator gate.

DGP	truth	passes H2 alone	passes full rule	gates, absolute	gates, conditional on H2
D1	M1	0.975	0.975	0.000	0% removed
D2	M1	0.420	0.420	0.000	0%
D3	M2	0.328	0.100	0.228	69%
D4	M1b	0.180	0.010	0.170	94%

The absolute cost of the comparator gates is therefore 0.17–0.23 of the shortfall on the two cells where they bind.

Both framings hold. In absolute terms the dominant failure is H2 – the true module often does not out-predict persistence on data it generated. Conditional on clearing H2, the comparator requirement is a dominant further filter.

SI-5.2 The chance baseline

With five structural modules, random assignment would place the generating module first 20% of the time.

DGP	true module has lowest h=1 RMSE	versus chance
D1 (M1 true)	0.627 (62.7%)	far better
D2 (M1 true)	0.645 (64.5%)	far better
D3 (M2 true)	0.0375	worse than chance
D4 (M1b true)	0.258 (25.8%)	barely better

On D3 the stock-flow module’s mean one-year error (85.7 kt) exceeds that of the residual (77.9 kt) and Allee (79.9 kt) modules. Estimation noise actively disadvantages the correct structure relative to its siblings at this sample size.

SI-5.3 Alternative decision rules

Computed on the same archived replicates, no refitting.

Decision rule	mean power	specificity
Retention rule as adopted (H1–H3, 5% band)	0.376	0.978
Without the comparator gate (H2, H3 only)	0.476	0.972
Persistence only, h=1, 5% band	0.562	0.955
Persistence only, any margin	0.542	0.765
Full rule, no tie band	0.436	0.772
Full rule, 10% band	0.337	0.998
MASE < 1 against the naive benchmark	0.651	0.675
Information criterion, $n \cdot \log \text{MSE} + 2k$	0.509	0.992

False-retention across the four structural DGPs is 0.044 per module-replicate pair, or 0.176 falsely retained modules per replicate; under the persistence-true null the per-module rate is 0.005.

SI-5.4 Two failed pre-check candidates

A diagnostic that told an analyst in advance whether the rule has power on a given series would be more valuable than any rule variant. Two were examined and neither works.

1. **Dispersion of scores across modules.** Median coefficient of variation against true-module power: Spearman $\rho = 0.52$. Too weak to license a protocol.
2. **Margin of the best-scoring module over persistence.** Spearman $\rho = 0.88$, Pearson $r = 0.94$ — strongly predictive, but it fails on two grounds. It is computed from the same out-of-sample scores the retention rule consumes, so it cannot gate the decision; and thresholding it at the tie band misclassifies both D3 cells, flagging “apply” where the generating module wins less often than chance.

Identifying when the rule has power therefore remains an open problem.

SI-6 Reproducibility package

Everything in the main text and in this supplement recomputes from the registered scripts and result files archived at <https://github.com/MIKEAA2020/general-sustainability>. The inventory below is the complete computational surface of the cod paper; checksums (md5, first 8 hex) pin the exact file states scored at release, and every entry is deterministic — re-execution in a fresh environment regenerates the archived outputs byte for byte.

Script (repo path)	Role in the paper	md5	
wave_e_cod/src/ run_ladder.py	Passes 1 (regime catch) and 2 (annual landings); model ladder scoring; writes meta.json keys catch_pass1 / catch_pass2	db6b4ae6	results fixed_window_scores.csv, rolling_summary.csv, meta.json
wave_e_cod/src/ run_xte.py	Pass 3: xteNCAM extended-series scoring (imports SPECS from run_ladder.py)	5a5229fd	results xte_* scores
wave_e_cod/src/ run_capelin_index.py	Capelin-index intervention module (scored against origin-matched persistence)	fa648aab	capelin index scores
wave_e_cod/src/ run_capelin_regime.py	Capelin wind-era variant	4fda8885	capelin regime scores
arena_agent_1/ other documents/ rerun_campaigns/campaign_e1_baselines.py	Recomputed naive baselines, origin-matched, asserted against frozen values	0159acd0	verified baseline print-outs
arena_agent_1/ other documents/ rerun_campaigns/campaign_e1_m1_reconciliation.py	M1 fit reconciliation audit	a97aed57	reconciliation output
batch 7 (audits of agent arena 1 paper rewrites)/campaign_e1_dm_uncertainty.py	Post-freeze DM + moving-block bootstrap layer (Spec A/B), seed 0, 20,000 replications, deterministic	81c559b8	results/ e1_dm_uncertainty.csv
batch 7 (audits of agent arena 1 paper rewrites)/campaign_e1_dm_uncertainty_regime.py	Regime-stratified DM layer	6ced3555	results/e1_dm_uncertainty_regime.csv

Execution order. run_ladder.py (Pass 1 then Pass 2) → run_xte.py (Pass 3) → run_capelin_index.py / run_capelin_regime.py (Pass 4) → campaign_e1_dm_uncertainty.py and campaign_e1_dm_uncertainty_regime.py (post-freeze audit layers, read-only on scores). campaign_e1_baselines.py and campaign_e1_m1_reconciliation.py are audit reruns: they reassert the frozen values, they never re-decide them.

Dependency claim. No script requires external credentials; the only non-redistributable raw input is the DFO assessment table itself, whose registered transcription is archived as wave_e_cod/data/ncam_2016_table_a2.csv and wave_e_cod/data/xtencam_table17_ssb.csv.